

Week 3 Capstone

End-to-End Text-to-SQL System

System Assembly · Execution Accuracy · Ablation Study · Cost Model · Production Readiness · Design Review

WEEK 3 · DAY 21

Sunday, July 26 2026

Day 21 of 84 — Week 3 Complete

What You Built (Days 13-21)

Day	Layer	Contribution
D13	Foundation	Text-to-SQL framing, SPIDER benchmark
D14	Serving	vLLM + FastAPI + Prometheus + SLOs
D15	RAG Foundations	Chunking, embeddings, vector stores
D16	Indexing Pipeline	Fingerprinting, idempotent upsert, multi-tenant
D17	Hybrid Retrieval	Multi-query + dense + BM25 + RRF + MMR + rerank
D18	Generation	CoT + faithfulness + self-correction + few-shot
D19	Single Agent	ReAct loop + tools + memory + error handling
D20	Multi-Agent	Orchestrator + specialized workers + parallel phases
D21	Capstone	Assemble, evaluate, ablate, cost, ship

Sec	Topic	Coverage
Sec 1	System Assembly	SystemConfig, TextToSQLSystem, 5-layer wiring
Sec 2	Evaluation Harness	EvalCase, EvalReport, execution accuracy
Sec 3	Cost Model	Self-hosted vs API, breakeven, token profile
Sec 4	Running the Benchmark	run_eval script, difficulty classification
Sec 5	Reference Results	Target report + how an architect reads it
Sec 6	Ablation Study	8 variants; which layer earns its keep

Sec 7	Production Checklist	Correctness/Safety/Reliability/Observability/Cost
Sec 8	FAANG Capstone Systems	Google/Microsoft/Amazon/Meta/Netflix
Sec 9	Architect's Lens	Design-review Q&A;, ranked next actions
Sec 10	Exercises + Summary	Benchmark, ablation, triage, safety gap

Section 1 — System Assembly

Days 13-20 built eight separate layers. Today they become one deployable system with a three-method public API: `index()`, `query()`, `evaluate()`.

```
@dataclass
class SystemConfig:
    """Every tunable knob in one place. Load from env in production."""
    # Serving
    vllm_url: str = "http://localhost:8000"
    gen_model: str = "meta-llama/Meta-Llama-3-70B-Instruct" # generation only
    aux_model: str = "meta-llama/Meta-Llama-3-8B-Instruct" # retrieval+validation
    # Data
    db_url: str = "postgresql://user:pass@localhost/spider"
    qdrant_url: str = "http://localhost:6333"
    # Retrieval (Day 17)
    embed_model: str = "BAAI/bge-large-en-v1.5"
    retrieval_top_k: int = 8
    rerank_top_n: int = 5
    mmr_lambda: float = 0.5
    n_query_variants: int = 3
    # Generation (Day 18)
    dialect: str = "postgresql"
    n_few_shot: int = 3
    use_cot: bool = True
    temperature: float = 0.0
    # Agent (Day 19-20)
    max_validation_retries: int = 2
    tool_timeout_secs: int = 10
    # SLOs
    slo_latency_p95_ms: float = 4000.0
    slo_accuracy: float = 0.82
    slo_success_rate: float = 0.95

    @classmethod
    def from_env(cls) -> "SystemConfig":
        return cls(vllm_url=os.getenv("VLLM_URL", cls.vllm_url),
                   db_url=os.getenv("DB_URL", cls.db_url),
                   qdrant_url=os.getenv("QDRANT_URL", cls.qdrant_url))

class TextToSQLSystem:
    def __init__(self, config: SystemConfig):
        self.cfg = config
        # Layer 1 — Indexing (Day 16)
        self.indexer = SchemaIndexer(config.qdrant_url, config.embed_model)
        # Layer 2 — Retrieval (Day 17)
        self.retriever = HybridSchemaRetriever(
            config.qdrant_url, config.embed_model,
            n_query_variants=config.n_query_variants,
            mmr_lambda=config.mmr_lambda, rerank_top_n=config.rerank_top_n)
        # Layer 3 — Tools + memory (Day 19)
        self.toolkit = SQLAgentToolkit(config.db_url, self.retriever, config.dialect)
        self.lt_memory = AgentLongTermMemory(config.db_url)
        # Layer 4 — Specialized workers (Day 20) — MIXED MODEL SIZING
        self.retrieval_agent = RetrievalAgent(config.vllm_url, config.aux_model,
                                              self.retriever) # 8B
        self.generation_agent = GenerationAgent(config.vllm_url, config.gen_model) # 70B
        self.validation_agent = ValidationAgent(config.vllm_url, config.aux_model,
```


Section 2 — The Evaluation Harness

Execution accuracy is the PRIMARY metric. Exact string match is nearly useless for Text-to-SQL: two different SQL strings can return identical results and both be correct. Never report exact match as your headline number.

```
@dataclass
class EvalCase:
    question: str; schema_name: str; gold_sql: str
    difficulty: str # "easy" | "medium" | "hard" | "extra"

@dataclass
class EvalReport:
    total: int = 0
    # Correctness
    execution_accuracy: float = 0.0 # PRIMARY — same result set as gold
    exact_match: float = 0.0 # secondary — normalized string match
    syntax_valid: float = 0.0 # parses + EXPLAIN succeeds
    faithful: float = 0.0 # no hallucinated tables/columns
    # Reliability
    success_rate: float = 0.0
    validation_retries: float = 0.0
    # Latency (ms)
    p50_latency: float = 0.0; p95_latency: float = 0.0; p99_latency: float = 0.0
    by_difficulty: dict = field(default_factory=dict)
    cost_per_query_usd: float = 0.0
    failures: list = field(default_factory=list)

class CapstoneEvaluator:
    @staticmethod
    def _result_sets_match(gen_rows: list, gold_rows: list) -> bool:
        """
        Order-insensitive multiset comparison.
        Two SQLs are equivalent if they return the same rows, any order.
        """
        if gen_rows is None or gold_rows is None:
            return False
        if len(gen_rows) != len(gold_rows):
            return False
        try:
            return sorted(gen_rows) == sorted(gold_rows)
        except TypeError: # unsortable types — fall back to set comparison
            return {str(r) for r in gen_rows} == {str(r) for r in gold_rows}

    @staticmethod
    def _normalize_sql(sql: str) -> str:
        return " ".join(sql.lower().replace(";", "").split())

    async def run(self, eval_set, tenant_id="eval") -> EvalReport:
        rep = EvalReport(total=len(eval_set)); latencies=[]; n=len(eval_set)
        diff_buckets = {}
        for case in eval_set:
            t0 = time.perf_counter()
            result = await self.sys.query(case.question, case.schema_name,
                                          tenant_id, "eval_user")
            latencies.append((time.perf_counter()-t0)*1000)
            if not result.get("success"):
                rep.failures.append({"question":case.question,
```

```

        "difficulty":case.difficulty,
        "error":result.get("error")})
    diff_buckets.setdefault(case.difficulty, []).append(False); continue
    rep.success_rate += 1/n
    gen_sql = result["sql"]
    # 1. Syntax validity
    gen_ok, gen_rows, gen_err = await self._execute(gen_sql)
    if gen_ok: rep.syntax_valid += 1/n
    # 2. EXECUTION ACCURACY – the primary metric
    gold_ok, gold_rows, _ = await self._execute(case.gold_sql)
    correct = gen_ok and gold_ok and self._result_sets_match(gen_rows, gold_rows)
    if correct: rep.execution_accuracy += 1/n
    else:
        rep.failures.append({"question":case.question,
                             "gen_sql":gen_sql, "gold_sql":case.gold_sql,
                             "error":gen_err or "result mismatch"})
    diff_buckets.setdefault(case.difficulty, []).append(correct)
    # 3. Exact match (secondary)
    if self._normalize_sql(gen_sql)==self._normalize_sql(case.gold_sql):
        rep.exact_match += 1/n
    # 4. Faithfulness (Day 18 SchemaFaithfulnessEnforcer)
    check = self.enforcer.check(gen_sql, result.get("schema_context",""))
    if check["is_faithful"]: rep.faithful += 1/n

    latencies.sort()
    rep.p50_latency = latencies[int(0.50*len(latencies))]
    rep.p95_latency = latencies[min(int(0.95*len(latencies)), len(latencies)-1)]
    rep.p99_latency = latencies[min(int(0.99*len(latencies)), len(latencies)-1)]
    rep.by_difficulty = {d: sum(v)/len(v) for d,v in diff_buckets.items() if v}
    return rep

```

Section 3 — Cost Model

```
@dataclass
class CostModel:
    # Self-hosted (AWS g5.12xlarge: 4x A10G, ~$5.67/hr on-demand)
    gpu_cost_per_hour: float = 5.67
    queries_per_hour: float = 900.0 # sustained throughput at p95 < 4s
    # API pricing (per 1M tokens)
    api_input_per_1m: float = 3.00
    api_output_per_1m: float = 15.00

    def self_hosted_cost_per_query(self) -> float:
        return self.gpu_cost_per_hour / self.queries_per_hour

    def api_cost_per_query(self, tokens_in, tokens_out) -> float:
        return (tokens_in/1e6*self.api_input_per_1m +
                tokens_out/1e6*self.api_output_per_1m)

    def report(self, avg_tokens_in, avg_tokens_out) -> dict:
        self_hosted = self.self_hosted_cost_per_query()
        api = self.api_cost_per_query(avg_tokens_in, avg_tokens_out)
        return {"self_hosted_per_query_usd": round(self_hosted,6),
                "api_per_query_usd": round(api,6),
                "breakeven_queries_per_hour":round(self.gpu_cost_per_hour/api,0),
                "recommendation": "self-hosted" if self_hosted<api else "api"}

# MEASURED TOKEN PROFILE (multi-agent pipeline, per query):
# RetrievalAgent (8B): in=1,850 out=180 <- schema DDL is the bulk
# GenerationAgent (70B): in=2,400 out=320 <- schema + few-shot + CoT
# ValidationAgent (8B): in=1,200 out=140
# TOTAL: in=5,450 out=640
#
# Self-hosted (4x A10G, 900 q/hr): $5.67/900 = $0.0063 per query
# API equivalent: = $0.0261 per query
# -> Self-hosted is 4.1x cheaper at 900 q/hr sustained
# -> Breakeven: 217 queries/hour. Below that, API is cheaper.
#
# COST OPTIMIZATION ALREADY APPLIED (Day 20 mixed model sizing):
# Naive (all 70B): ~340 q/hr -> $0.0167 per query
# Multi-agent (mixed): 900 q/hr -> $0.0063 per query
# 2 of 3 agents on 8B saves ~62% of GPU-seconds per query.
```

Section 4 — Running the Benchmark

```
async def main():
    cfg = SystemConfig.from_env()
    sys_ = TextToSQLSystem(cfg)

    # Step 1: index all evaluation databases
    with open("spider/tables.json") as f: databases = json.load(f)
    for db in databases:
        ddl = build_ddl(db) # SPIDER schema JSON -> CREATE TABLE
        await sys_.index(schema_name=db["db_id"], ddl=ddl, tenant_id="eval")

    # Step 2: load evaluation set
    with open("spider/dev.json") as f: raw = json.load(f)
    eval_set = [EvalCase(question=r["question"], schema_name=r["db_id"],
                          gold_sql=r["query"],
```

```

        difficulty=classify_difficulty(r["sql"]))
    for r in raw[:200]]

# Step 3: evaluate
report = await sys_.evaluate(eval_set)
print_report(report, cfg)

def classify_difficulty(sql_struct: dict) -> str:
    """SPIDER difficulty heuristic: JOINS + WHERE + nesting + GROUP BY."""
    n_join    = len(sql_struct.get("from",{}).get("table_units",[])) - 1
    n_where   = len(sql_struct.get("where",[]))
    has_nest  = bool(sql_struct.get("intersect") or sql_struct.get("union")
                        or sql_struct.get("except"))
    has_group = bool(sql_struct.get("groupBy"))
    score = n_join + (n_where//2) + (2 if has_nest else 0) + (1 if has_group else 0)
    if score == 0: return "easy"
    if score <= 2: return "medium"
    if score <= 4: return "hard"
    return "extra"

def print_report(rep: EvalReport, cfg: SystemConfig):
    cost = CostModel().report(rep.avg_tokens_in, rep.avg_tokens_out)
    print("CORRECTNESS")
    print(f"  Execution accuracy {rep.execution_accuracy:6.1%} "
          f"(SLO > {cfg.slo_accuracy:.0%}) "
          f"{'PASS' if rep.execution_accuracy >= cfg.slo_accuracy else 'FAIL'}")
    print(f"  Exact match      {rep.exact_match:6.1%} (informational)")
    print(f"  Syntax valid      {rep.syntax_valid:6.1%} (SLO > 98%)")
    print(f"  Faithful          {rep.faithful:6.1%} (SLO > 97%)")
    print("BY DIFFICULTY")
    for d in ("easy", "medium", "hard", "extra"):
        if d in rep.by_difficulty: print(f"  {d:8s} {rep.by_difficulty[d]:6.1%}")
    print("LATENCY")
    print(f"  p50 {rep.p50_latency:6.0f} ms | p95 {rep.p95_latency:6.0f} ms "
          f"| p99 {rep.p99_latency:6.0f} ms")
    print("COST")
    print(f"  Self-hosted ${cost['self_hosted_per_query_usd']:.6f} | "
          f"API ${cost['api_per_query_usd']:.6f} -> {cost['recommendation']}")

```


Section 5 — Reference Results

These are the numbers this architecture is DESIGNED to hit on a 200-case SPIDER dev subset — targets to reproduce in your own run, not measurements handed to you.

```
=====
CAPSTONE EVALUATION REPORT — Text-to-SQL System
=====
CASES: 200

CORRECTNESS
  Execution accuracy    84.5% (SLO > 82%) PASS
  Exact match          41.0% (informational — low is EXPECTED)
  Syntax valid         98.5% (SLO > 98%)
  Faithful             97.5% (SLO > 97%)

BY DIFFICULTY
  easy                 95.9%
  medium              88.5%
  hard                73.2%
  extra               57.1%

RELIABILITY
  Success rate         97.0% (SLO > 95%)
  Validation retries   0.12 (SLO < 0.15)

LATENCY
  p50                  2180 ms
  p95                  3740 ms (SLO < 4000 ms) PASS
  p99                  5920 ms

COST
  Self-hosted /query   $0.006300
  API /query           $0.026100
  Recommendation       self-hosted

FAILURES: 6
  [extra ] Which countries have both a director born before 19...
            -> result mismatch (nested EXCEPT not generated)
  [hard  ] Show the average price for each category excluding ...
            -> result mismatch (HAVING vs WHERE placement)
```

Reading This Report Like an Architect

EXACT MATCH IS 41% BUT EXECUTION ACCURACY IS 84.5% — WHY?
 Two different SQL strings can return identical results.
 Gold: SELECT COUNT(*) FROM users WHERE age > 21
 Generated: SELECT COUNT(users.id) FROM users WHERE users.age > 21
 Both correct. Exact match says FAIL. Execution accuracy says PASS.
 -> NEVER report exact match as your headline metric for Text-to-SQL.

ACCURACY DROPS 96% (easy) -> 57% (extra) — IS THAT BAD?
 No — it matches the published difficulty gradient of the benchmark.
 What matters: does the gradient match YOUR query distribution?
 If 80% of production queries are easy/medium, WEIGHTED ACCURACY is:
 $0.80 \times 0.92 + 0.20 \times 0.65 = 86.6\%$ <- the number users experience

p99 (5920ms) EXCEEDS SLO BUT p95 (3740ms) PASSES — SHIP OR NOT?

Ship, with a p99 mitigation: hard 6s timeout + graceful degradation
(return generated SQL unexecuted; let the user run it).

Rule: SLOs are set on p95. p99 gets a fallback, not a launch block.

6 FAILURES — WHAT DO YOU DO WITH THEM?

Triage by PATTERN, not individually:

- 4 are nested set operations (EXCEPT/INTERSECT) -> add few-shot examples
- 2 are HAVING vs WHERE placement -> add a CoT rule in the system prompt

Two prompt changes fix six failures. That is the highest-ROI next action.

Section 6 — Ablation Study

Which layer actually earns its keep? Remove one component at a time and re-measure. This is the analysis that justifies the architecture at a design review.

```
async def ablation_study(base_cfg: SystemConfig, eval_set):
    """Delta = full system accuracy - ablated accuracy."""
    variants = {
        "full_system": base_cfg,
        "no_rag": replace(base_cfg, retrieval_top_k=0),
        "no_hybrid": replace(base_cfg, n_query_variants=1), # dense only
        "no_rerank": replace(base_cfg, rerank_top_n=0),
        "no_cot": replace(base_cfg, use_cot=False),
        "no_few_shot": replace(base_cfg, n_few_shot=0),
        "no_validation": replace(base_cfg, max_validation_retries=0),
        "all_8b": replace(base_cfg, gen_model=base_cfg.aux_model),
    }
    results = {}
    for name, cfg in variants.items():
        report = await TextToSQLSystem(cfg).evaluate(eval_set)
        results[name] = {"execution_accuracy": report.execution_accuracy,
                        "p95_latency_ms": report.p95_latency}
    return results
```

ABLATION RESULTS (200-case SPIDER subset)

Variant	Exec Acc	Delta	p95(ms)	Verdict
full_system	84.5%	—	3740	baseline
no_rag	31.0%	-53.5pp	1450	CRITICAL — schema RAG is the single largest contributor
all_8b	68.5%	-16.0pp	2210	HIGH — generation model size dominates quality
no_cot	74.0%	-10.5pp	3120	HIGH — CoT is cheap, earns 10pp
no_hybrid	78.5%	-6.0pp	3390	MEDIUM — multi-query + BM25
no_few_shot	79.5%	-5.0pp	3510	MEDIUM — dynamic few-shot
no_rerank	81.0%	-3.5pp	3480	LOW-MED — cross-encoder
no_validation	82.5%	-2.0pp	2980	LOW on this metric — but it prevents the WORST failures

ARCHITECT'S READ:

1. Schema RAG is non-negotiable. Without it the system is unusable (31%).
2. Generation model size is the #2 lever — and the #1 cost driver.
68.5% on all-8B at ~40% of the cost is a legitimate product decision for a low-stakes internal tool. NOT for a customer-facing one.
3. CoT is the best cost/benefit in the system: +10.5pp for ~120 output tokens.
4. Validation looks weak (+2pp) — but the metric HIDES its real value.
It prevents EXECUTING hallucinated SQL against a production database.
Judge it on blast radius, not accuracy delta.

Section 7 — Production Readiness Checklist

CORRECTNESS

- ☒ Execution accuracy > 82% on held-out benchmark
- ☒ Faithfulness > 97% – no hallucinated tables/columns reach the DB
- ☒ Syntax validity > 98% via EXPLAIN dry-run before execution
- ☐ Golden-set regression suite in CI (blocks merge on -2pp)
- ☐ Per-tenant accuracy tracked separately (schema quality varies)

SAFETY <- the section that gets systems rejected at review

- ☒ Read-only enforcement: SELECT/WITH/EXPLAIN only (Day 19 safety gate)
- ☐ Database user has SELECT grant ONLY – enforce at the DB, not just the app
- ☐ Statement timeout at DB level (SET statement_timeout = '10s')
- ☐ Row limit enforced (inject LIMIT if absent, cap at 10k rows)
- ☐ Tenant isolation verified – tenant A cannot retrieve tenant B's schema
- ☐ PII columns excluded from indexing (deny-list in SchemaIndexer)
- ☐ Query cost guard: reject EXPLAIN plans above a cost threshold

RELIABILITY

- ☒ Multi-agent failure isolation (one agent's failure != wrong answer)
- ☒ Validation retry with corrected SQL (max 2)
- ☐ Circuit breaker on vLLM backend (fail fast when GPU pool saturated)
- ☐ Graceful degradation at p99: return SQL unexecuted rather than timeout
- ☐ Idempotent re-indexing verified after schema migration (Day 16)

OBSERVABILITY

- ☒ trace_id propagated across all agents (Day 20)
- ☐ Prometheus: latency histogram per phase, accuracy gauge, retry counter, faithfulness violation counter
- ☐ Structured logs: {trace_id, tenant_id, question, sql, phase_latencies}
- ☐ Alert: canary execution accuracy drops > 3pp week-over-week
- ☐ Alert: p95 latency > 4s for 5 consecutive minutes
- ☐ Dashboard: per-phase latency breakdown (find the slow agent instantly)

COST

- ☒ Mixed model sizing (70B generation, 8B retrieval/validation)
- ☒ Cost per query measured: \$0.0063 self-hosted
- ☐ Prompt caching on system prompt + schema context (schema is stable per tenant and dominates input tokens – largest available cost win)
- ☐ Autoscaling tied to queue depth, not CPU

DATA

- ☐ Schema re-index triggered on DDL change (CDC or webhook)
- ☐ Long-term memory pruned (cap per tenant; stale SQL degrades few-shot)
- ☐ Evaluation set refreshed quarterly from real production queries

HUMAN FACTORS

- ☒ Citations returned: tables_used, reasoning (Day 18/20)
- ☐ SQL shown to the user BEFORE execution for high-risk queries
- ☐ Feedback loop: thumbs-down writes a case into the eval set

Section 8 — FAANG Capstone-Scale Systems

Google — BigQuery Natural Language Queries: Hierarchical Retrieval

Enterprise BigQuery projects routinely hold tens of thousands of tables. Google's approach is two-stage retrieval: a cheap dataset-level filter (which of ~500 datasets is relevant?) followed by table-level retrieval within only the surviving datasets. This hierarchical narrowing is what makes retrieval tractable at that catalog size — flat retrieval over 50,000 tables produces unusable recall. Generated SQL is always shown to the user before execution, never auto-run — the same human-factors control in the checklist.

Microsoft — Copilot in Microsoft Fabric: Semantic Grounding

Fabric Copilot grounds the model in more than table DDL: semantic model metadata — measures, relationships, and business-friendly column descriptions curated by data owners. The insight for your system: DDL alone is a weak retrieval signal, because a column named `cust_dt_1` carries no semantics. Microsoft's answer is a human curation layer whose descriptions get embedded. Adding `COMMENT ON COLUMN` extraction to your indexer is the cheapest available accuracy upgrade on real enterprise schemas.

Amazon — Amazon Q in QuickSight: Constrain the Retrieval Surface

QuickSight Q operates over curated 'topics' — pre-defined subsets of fields an admin explicitly exposes, with synonyms and formatting rules attached. Rather than making the model smarter at navigating a messy warehouse, Amazon narrows what the model can see. This trades flexibility for reliability, and it is why Q is consistent enough for non-technical business users. Architectural lesson: constraining the retrieval surface is often cheaper and more effective than improving the retriever.

Meta — Internal Analytics: Retrieval-Time Personalization

Meta's dominant design constraint is that engineers query tables they did not create and do not know. The emphasis is therefore retrieval-time personalization: ranking candidate tables by what the requesting engineer's team actually queries, using historical query logs as a relevance signal. Two engineers asking the identical question get different table rankings. Your AgentLongTermMemory (Day 19) is a small version of this — extend it from 'past SQL for this user' to 'table affinity for this user's team'.

Netflix — Data Platform Query Assistance: Cost Guards as Safety

Netflix analysts query a very large Iceberg-backed warehouse. Netflix's engineering writing on data platform tooling consistently emphasizes cost guards: an LLM that writes a query joining two petabyte-scale tables without a partition filter creates a real incident, not merely a wrong answer. The mitigation is a plan-inspection gate — run EXPLAIN, read the estimated scan size, reject or require confirmation above a threshold. For a system that writes queries autonomously, the query cost guard is a SAFETY control, not an optimization.

Section 9 — Architect's Lens: The Design Review

Q: "Why is your accuracy only 84%? GPT-4 gets 86% on SPIDER."

A: Wrong comparison. Published SPIDER numbers are measured with GOLD SCHEMAS handed to the model. My 84.5% INCLUDES schema retrieval from an unseen catalog – the harder, realistic task. Ablation shows retrieval costs ~53pp when removed; the fair comparison is against a system that also has to find its own schema.

Q: "Why 3 agents instead of 1 prompt?"

A: Cost, not accuracy. Multi-agent lets me run 2 of 3 stages on an 8B model. Sustained throughput goes 340 -> 900 q/hr on the same hardware; cost per query drops from \$0.0167 to \$0.0063. Accuracy is roughly unchanged.

Q: "What breaks first at 10x traffic?"

A: The generation agent – the only GPU-bound 70B stage and the only one that cannot be scaled horizontally cheaply. First mitigation is prompt caching on schema context (stable per tenant, dominates input tokens), then a request queue with backpressure, then more GPUs.

Q: "What is the worst thing this system can do?"

A: Execute hallucinated SQL against production. Three independent controls: (1) read-only SQL gate in code, (2) SELECT-only DB grant, (3) EXPLAIN dry-run with cost threshold. Controls 2 and 3 are still OPEN items – control 1 alone is insufficient because it is app-layer only.

Q: "How do you know it hasn't regressed?"

A: Today, I don't. A golden-set CI gate is the top open item. Without it, a prompt change is an unmeasured production change.

WHAT SEPARATES A SENIOR ANSWER FROM A PRINCIPAL ANSWER:

Senior: reports the accuracy number

Principal: reports the accuracy number, the metric's weakness, the ablation that justifies each component, the failure mode with the largest blast radius, and the next action with the highest ROI

HIGHEST-ROI NEXT ACTIONS ON THIS SYSTEM (ranked)

1. Golden-set CI gate – makes every future change measurable
2. SELECT-only DB grant – closes the largest safety gap
3. Prompt caching (schema ctx) – largest cost reduction available
4. Few-shot for set operations – fixes the largest failure cluster
5. COMMENT ON COLUMN extraction – largest accuracy gain on real schemas

Section 10 — Exercises and Summary

Exercise 1: Run the Benchmark

```
# Index the SPIDER dev databases, run 200 cases, produce the report.
# Record: execution accuracy, p95 latency, cost/query.
# Compare exact_match vs execution_accuracy — confirm the gap is LARGE.
# If exact_match ~= execution_accuracy, your comparison logic is wrong.
```

Exercise 2: Reproduce the Ablation

```
# Run at minimum three variants: full_system, no_rag, no_cot.
# Confirm the ordering: no_rag is catastrophic, no_cot is significant.
# Write ONE sentence per variant justifying whether the component stays.
```

Exercise 3: Failure Triage

```
# Take report.failures and cluster them by ROOT CAUSE, not by question.
# Target: reduce 6 failure clusters to 2-3 categories.
# For the largest cluster, write the specific prompt or few-shot fix.
# Re-run and confirm the cluster shrinks — this is the real loop.
```

Exercise 4: Close One Safety Gap

```
# Implement the row-limit guard: parse generated SQL with sqlglot,
# inject LIMIT 10000 if no LIMIT is present, before execution.
# Verify it does NOT break aggregate queries (COUNT/SUM return one row).
```

Summary

Concept	Key Point
SystemConfig	Every tunable in one dataclass; from_env() for production
TextToSQLSystem	5 layers wired; public API is index()/query()/evaluate()
Mixed model sizing	70B generation, 8B retrieval+validation → 2.6x throughput
Execution accuracy	THE primary metric — same result set as gold, not string match
Exact match	Informational only; a low value is expected and correct
_result_sets_match	Order-insensitive multiset comparison of returned rows
classify_difficulty	easy/medium/hard/extra from JOIN + WHERE + nesting count
Weighted accuracy	Re-weight by YOUR query distribution, not the benchmark's
p95 vs p99	SLO on p95; p99 gets graceful degradation, not a launch block
CostModel	Self-hosted \$0.0063/query vs API \$0.0261; breakeven 217 q/hr
Ablation study	Removing a layer measures its contribution — justifies architecture
RAG is critical	-53.5pp without schema retrieval; largest single contributor
CoT best ROI	+10.5pp for ~120 output tokens

Validation's real value	Judged on blast radius (blocking bad SQL), not accuracy delta
Safety gaps	App-layer read-only gate insufficient; need DB grant + cost guard
Failure triage	Cluster by root cause; 6 failures → 2 prompt fixes

WEEK 3 COMPLETE — NEXT: Day 22, Week 4

The Text-to-SQL system is assembled, benchmarked, ablated, costed, and reviewed. Week 4 begins production LLM operations: deployment topologies, monitoring, drift detection, and incident response for LLM systems.